THE UNIVERSITY OF BAMENDA

NATIONAL HIGHER POLYTECHNIC INSTITUTE

DEPARTMENT OF COMPUTER ENGINEERING

PROJECT PROPOSAL

ZILA: A TERMINAL TOOL FOR AUTOMATED INTERNSHIP PROGRESS MONITORING AND LEARNING EVALUATION.

A Project Proposal Submitted to the Department of Computer Engineering in the National Higher Polytechnic Institute of The University of Bamenda in Partial Fulfilment of the Requirements for the Award of a Masters of Engineering Degree in Computer Engineering.

BY:

NGENGE JOHN RAWLINGS NSAME

REGISTRATION NUMBER: UBA25EP215

SUPERVISORS:

DR. SUH CHARLES FORBACHA

ENG. FOMONYUY JUDE GITA

APRIL 2026

ABSTRACT

When students go on internships, keeping track of what they are actually learning is a genuine challenge. Most students fill in a logbook at the end of each week, writing from memory what they did. Supervisors check these logbooks briefly and visit occasionally. This system is slow, easy to fill in dishonestly, and gives nobody a clear and continuous picture of how the student is progressing. This problem is not unique to any single platform or institution; it affects any program where students are placed in a technical work environment and assessed against a defined curriculum. This project proposes ZILA (Zigex Intelligent Layer for Agents), a terminal tool that runs quietly on a student's computer, monitors their git commits and file changes inside their internship working folder, and uses a large language model to map that activity to the internship curriculum automatically. At the end of each week, ZILA produces a plain language progress report showing both the student and their supervisor what was covered, what is missing, and what to focus on next, without the student having to write anything. ZILA is designed to work with any structured internship curriculum and has been built with integration support for the Zigex platform, developed by SEED, which serves as the reference implementation for this project. The project will build the tool, test whether its skill classifications match what human experts would say, check whether supervisors find the reports useful, and confirm that the tool runs lightly enough in the background not to slow down the student's computer.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Background of the Study 1
1.2 Statement of the Problem and Rationale 2
1.3 Hypotheses 3
1.4 Objectives 3
1.4.1 Overall Objective 3
1.4.2 Specific Objectives 3
CHAPTER 2: LITERATURE REVIEW 4
2.1 How Internship Progress Is Currently Assessed 4
2.2 What Developer Git Records Can Tell Us 4
2.3 Using AI to Understand Code and Classify Work 5
2.4 Background Monitoring Tools in Developer Environments 5
CHAPTER 3: METHODOLOGY 6
3.1 Phase 1: Building the Observation Layer (SO1) 6
3.2 Phase 2: Classification and Report Generation (SO2) 6
3.2.1 Connecting to an Internship Curriculum 7
3.2.2 Classifying Each Commit 7
3.2.3 Generating Weekly Reports 7
3.3 Phase 3: Evaluation (SO3) 8
3.3.1 Testing Classification Accuracy (H1) 8
3.3.2 Testing Report Usefulness (H2) 8
3.3.3 Testing System Performance (H3) 8
CHAPTER 4: EXPECTED OUTCOMES 9
4.1 What Will Be Produced 9
4.2 What the Tests Are Expected to Show 9
4.3 Broader Value 10
TIMEFRAME 10
BUDGET ESTIMATE 11
REFERENCES 12

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Internship placements are a central part of engineering education. They give students the opportunity to apply what they have learned in the classroom to real work, and at institutions including the National Higher Polytechnic Institute (NAHPI), completing an industrial attachment is a formal requirement for graduation. Yet despite how important internships are, the way student progress is tracked during them has not been improved significantly.

The two tools used most widely are the student logbook and the periodic supervisor visit. A student fills in their logbook which is usually at the end of the week, writing down what they did. A supervisor checks in occasionally and writes a short assessment. Both methods depend on people remembering and describing their work accurately after the fact. Studies of internship logbooks across engineering programs have found that most entries are written retrospectively, are vague about what was actually done, and do not clearly connect the work to what the student was supposed to be learning (Orr *et* al., 2018). Supervisor visits are infrequent, so a student can go weeks without any meaningful feedback. And when students at the same institution are supervised by different people, they may be assessed very differently for work of the same quality (Patrick *et* al., 2009).

This problem is not unique to one institution or one platform. It is a structural weakness of how internship learning is monitored across the sector. Any program that places technology students in a work environment and asks them to develop specific skills faces the same difficulty: there is no reliable, continuous mechanism for tracking whether learning is actually happening.

Software developers, unlike students in many other fields, naturally produce a record of their work as they go. Every time a developer saves a meaningful change to their code using a version control system like git, they create a small log entry: what files changed, what the changes were, and a short message describing the work. Researchers who have studied large collections of these records have shown that they carry real information about what a developer was working on and what kinds of skills they were exercising (Kalliamvakou et al., 2014). If a tool could read those records automatically and connect

them to what the student was supposed to be learning, it could produce a much more accurate and continuous picture of progress than any logbook.

Recent progress in AI language models makes this possible. These models can read code and plain-text descriptions, understand what the code is doing, and connect it to a topic or skill area. When such a model is given the internship curriculum as a reference - a structured list of what the student should be learning, it can compare each piece of work against that list and identify what has been covered and what has not. Lewis et al. (2020) showed that giving an AI model access to an external reference at the time it makes a decision - a technique called Retrieval-Augmented Generation, or RAG. This significantly improves the accuracy of its outputs compared to asking it to work from memory alone. This is the technical foundation that makes ZILA feasible.

1.2 Statement of the Problem and Rationale

There is tool that can automatically observe what a student intern is working on, understand what that work means in terms of the skills they are supposed to be developing, and produce a regular, structured progress report - without requiring the student to write or report anything themselves.

This gap has three practical consequences. First, supervisors only find out how a student is progressing when they check in or read the logbook. If a student has been avoiding a topic for several weeks, nobody knows until much later. Second, students themselves rarely have a clear picture of where they are falling behind until a formal assessment at the end of the placement. Third, the logbook is easy to fill in dishonestly - a student who has had an unproductive week can write entries that make it appear otherwise, and there is usually no way to verify them.

ZILA addresses this problem by monitoring a student's actual work output by looking at their git commits and file changes, and using AI to interpret that work against a structured internship curriculum. It then generates a weekly progress report automatically. The student does not have to fill in anything extra. The supervisor receives a clear, evidence based summary of what has been covered and what gaps remain.

ZILA is designed as a general tool, not a platform-specific one. The internship curriculum it works with is provided through a configurable integration layer. For this project, that integration has been built for the Zigex platform - an internship and opportunities portal developed by SEED, the company hosting this industrial attachment. Zigex serves as the

reference implementation: it is the platform against which ZILA is built and evaluated. However, the core system is not dependent on Zigex and could be connected to any platform that provides a structured curriculum in a compatible format.

1.3 Hypotheses

Three specific, measurable hypotheses guide the design and evaluation of this project:

H1 - Classification Accuracy: When ZILA reads a student's git commit and classifies it against an internship curriculum, it will correctly identify the relevant skill area at least 75% of the time, as measured against labels agreed upon by human expert reviewers.

H2 - Report Usefulness: At least 70% of supervisors and academic staff who review ZILA generated weekly progress reports will rate them as accurate and useful enough to support internship evaluation, using a structured rating questionnaire.

H3 - System Overhead: The ZILA background monitoring process will use less than 5% of average CPU capacity and less than 100 MB of memory during a normal student work session, confirming that it does not slow down or disrupt the student's computer.

1.4 Objectives

1.4.1 Overall Objective

To design, build, and evaluate ZILA - a background terminal tool that monitors a student intern's git commits and file changes, maps that activity to a structured internship curriculum using AI, and produces weekly progress reports for the student and their supervisor - with the Zigex platform serving as the reference implementation.

1.4.2 Specific Objectives

- **SO1:** To build the observation layer - a background process that detects and records a student's git commits and file edits inside their internship working folder, requiring minimal action from the student.
- **SO2:** To build the reporting engine - an AI pipeline that reads each commit, matches it to the internship curriculum via a configurable integration layer, and generates a structured weekly progress report showing coverage and gaps.
- **SO3:** To evaluate the system - measuring how accurately the AI classifies commits compared to human expert labels, whether supervisors find the generated

reports useful, and whether the tool runs lightly enough not to affect the student's working environment.

CHAPTER 2: LITERATURE REVIEW

2.1 How Internship Progress Is Currently Assessed

Work integrated learning is a broad category that covers internships, industrial attachments, and cooperative education has been studied widely across engineering and vocational education. A consistent finding is that student self-reporting, despite being the most common assessment method, produces unreliable evidence. (Coll *et* al., 2011) reviewed assessment practices across work integrated learning programs globally and found that logbooks and journals remained the primary tool even where their weaknesses were well understood. (Patrick *et* al., 2009), in a study of work-integrated learning, found that the quality of supervision and feedback students received varied widely depending on the individual supervisor which created a situation where a student's assessed outcome was shaped more by who supervised them than by what they actually did.

(Orr *et* al., 2018) examined student logbooks from four engineering faculties and found that most entries were written after the fact, described activities in vague terms, and failed to connect the work done to the learning outcomes the placement was designed to develop. These problems are not specific to any one institution or country. They reflect a structural weakness in how internship learning is monitored across the sector, and they provide the primary motivation for a more automated and evidence-based approach.

2.2 What Developer Git Records Can Tell Us

Software developers who use version control systems like git produce a structured record of their work as a natural byproduct of how they work. Each commit contains the files changed, the lines added or removed, a timestamp, and a message written by the developer. (Kalliamvakou *et* al., 2014) conducted a large-scale study of public software repositories and showed that these records carry meaningful information about developer activity including what kinds of tasks developers were working on and how they engaged with different technologies. While their study focused on professional open-source contributors, the same principle applies to student interns: git commit records are evidence of real work.

In educational settings, (Hsiao *et* al., 2015) showed that patterns in student code submissions could be analyzed automatically to detect engagement and identify struggling students, without requiring those students to report anything. Their system used rule-based analysis, which limited what it could infer. ZILA extends this by using a language model to perform richer, semantic interpretation of the same kinds of records - enabling it to identify not just whether a student was active, but what they were working on and how that maps to a curriculum.

2.3 Using AI to Understand Code and Classify Work

Large language models have demonstrated a strong ability to read and understand code. (Ahmad *et* al., 2021) showed that a model trained on both code and natural language could accurately describe what a code change does - essentially the reverse of what ZILA needs to do. If a model can describe a code change, it can equally be asked to categorize what skill area that change relates to.

A known risk with these models is that they can produce plausible sounding but incorrect outputs, particularly when asked about topics not well covered in their training data. (Lewis *et* al., 2020) addressed this with Retrieval-Augmented Generation (RAG), in which the model is given relevant reference material at the time it makes a decision, rather than relying on memory alone. In ZILA, this reference material is the internship curriculum: when classifying a commit, the model is shown the actual curriculum descriptors for the skill areas most likely to be relevant, which keeps its output grounded in what the placement program actually requires. This approach works regardless of which curriculum is used, because the curriculum content is retrieved at runtime rather than baked into the model.

2.4 Background Monitoring Tools in Developer Environments

Tools that run passively in the background and observe developer activity already exist and are accepted by users. (WakaTime, 2013), for example, watches which files a developer edit and for how long, producing summaries of how time was spent across different projects and languages. This demonstrates that lightweight, continuous observation of developer activity is both technically feasible and practically acceptable, provided the tool is transparent about what it collects and the data is used to benefit the developer.

What does not currently exist is a tool that connects this kind of passive observation to a formal learning curriculum and produces structured educational assessments from it. ZILA occupies this space - combining the observation model of tools like WakaTime with AI-driven curriculum mapping to turn everyday developer activity into evidence of learning progress.

CHAPTER 3: METHODOLOGY

The project is built across three phases, each corresponding directly to one of the specific objectives. Each phase produces outputs that are tested against the hypotheses.

3.1 Phase 1: Building the Observation Layer (SO1)

This phase builds the part of ZILA that watches what the student is doing. When a student runs zila init inside their internship working folder, two things happen automatically.

First, ZILA installs a post-commit hook into the folder's git repository. A post-commit hook is a small script that git runs automatically every time the student makes a commit. ZILA's hook captures the commit message, the list of files changed, and the actual content of those changes, and saves them as a structured record in a local database on the student's machine. The student does not have to do anything differently - the hook runs silently each time they commit.

Second, ZILA starts a lightweight background process, written in Rust for its low resource footprint, that watches the working folder for file changes. When a file is saved or created, the process logs which file changed and when. These file change records provide supporting context alongside the commit records, capturing active work periods that have not yet been committed.

At the end of each day, ZILA combines the day's commit records and file change records into a single daily activity summary. Commit records are treated as the primary evidence, because they are intentional - the student has explicitly packaged and described a piece of completed work. File changes without associated commits are treated as supplementary context.

3.2 Phase 2: Classification and Report Generation (SO2)

This phase builds the part of ZILA that makes sense of what was observed and produces the progress reports.

3.2.1 Connecting to an Internship Curriculum

ZILA is designed to work with any structured internship curriculum provided through a compatible integration. At setup, the student connects ZILA to their curriculum source - for this project, the Zigex platform, which provides each student's curriculum through an API. ZILA downloads the curriculum at initialization, stores it locally, and prepares it for fast searching. The curriculum is a structured list of skill areas and learning outcomes, each described in plain language. The same setup process would work with any other platform or even a locally stored curriculum file, provided the content follows the same schema.

3.2.2 Classifying Each Commit

Each time a new commit record is processed, ZILA searches the stored curriculum to find the skill areas most relevant to that commit. It then sends those curriculum descriptions, along with the commit message and the files changed, to a language model (Mistral-7B via the Open Router API, with a local Llama model as an offline alternative). The model is asked to identify which curriculum skill area the commit most closely relates to and to give a brief, plain-language explanation of why. The model's response is required in a fixed structured format, which prevents vague or off-topic answers.

3.2.3 Generating Weekly Reports

At the end of each week, ZILA collects all the classified commit records for the past seven days. It calculates which curriculum skill areas were covered, which were not touched at all, and which were only briefly engaged with. It then uses the language model to write a short, plain-language weekly progress report following a fixed template: a summary of the week's activity, a breakdown by curriculum area, a list of areas not yet covered, and a suggested focus for the coming week. This report is saved locally and, through the platform integration, made available to the student's supervisor.

3.3 Phase 3: Evaluation (SO3)

3.3.1 Testing Classification Accuracy (H1)

To measure how accurately ZILA classifies commits, a dataset of at least 200 real git commits from student internship repositories will be collected, with the students' permission. Two human experts - engineers from SEED and academic staff from NAHPI - will independently read each commit and label it with the curriculum skill area it represents. Where the two experts disagree, they will discuss and reach a consensus; commits where consensus cannot be reached will be set aside. This produces a reference set of correctly labelled commits.

ZILA's classification pipeline will then be run on the same commits, and its labels compared to the reference set. The main measure is accuracy - what percentage of commits did ZILA label correctly? The target is 75%. The test will also be run with the curriculum retrieval step turned off, so that the language model has to classify without seeing the curriculum descriptions. Comparing the two results will show how much the curriculum grounding step contributes to accuracy.

3.3.2 Testing Report Usefulness (H2)

Between five and eight participants - a mix of SEED engineers and NAHPI academic supervisors - will be shown a set of ZILA-generated weekly progress reports produced from the annotated commit dataset. Each participant will rate each report on four questions: Is the activity described accurate? Is the report complete? Is it clearly written? Is it useful enough to support an assessment of the student's progress? Ratings use a five-point scale from strongly disagree to strongly agree. Participants may also write comments identifying anything they believe was wrong or missing. The target is that at least 70% of reports are rated as useful overall.

3.3.3 Testing System Performance (H3)

The ZILA background process will be run on a laptop representative of a student at NAHPI - an Intel Core i5 with 8 GB of RAM. Over ten separate one-hour sessions, CPU use and memory consumption will be recorded every second. Sessions will cover both quiet periods and active working periods. The target is that average CPU use stays below 5% and memory use stays below 100 MB throughout.

CHAPTER 4: EXPECTED OUTCOMES

4.1 What Will Be Produced

The main deliverable is a working version of ZILA, released as an open-source tool installable from the command line. It will include the background monitoring process, the AI pipeline, the weekly report engine, and the Zigex platform integration. A mock version of the curriculum API will be included so that the tool can be developed and tested independently of any live platform. The annotated commit dataset produced during the evaluation will also be published, so that other researchers can use it as a benchmark for similar work.

4.2 What the Tests Are Expected to Show

For H1, the curriculum-grounded classification approach is expected to meet the 75% accuracy target. The comparison run without curriculum retrieval is expected to show a meaningfully lower accuracy, which would confirm that grounding the model in the actual curriculum content - rather than relying on its general training knowledge - makes a practical difference to the quality of the output.

For H2, the main risk is not inaccuracy but incompleteness. ZILA only sees what a student does inside their git repository. Reading, attending meetings, verbal discussions, and other forms of learning are not captured. The reports will make this limitation explicit - they describe what is observable from code activity, not everything a student learned. With that caveat clear, the expectation is that supervisors will find the reports useful as a structured, evidence-based complement to their own observations, and that the 70% usefulness target will be met.

For H3, passive file-watching tools of this kind are typically very lightweight. The resource targets of under 5% CPU and 100 MB memory are expected to be met comfortably. The weekly report generation step uses more resources, but it is scheduled to run during inactive periods and should not cause any noticeable disruption to the student.

4.3 Broader Value

If ZILA works as expected, it is useful to three groups immediately. Students receive honest, regular feedback on their learning without any additional reporting burden. Supervisors receive structured, evidence-based summaries they can use to guide assessments and evaluations. SEED and Zigex receive a production-quality open-source tool that adds direct value to their platform.

Beyond this project, ZILA establishes a reusable pattern for automated learning progress monitoring in any code-based technical placement. Because the curriculum integration is configurable, the same tool could be adapted for university project modules, coding bootcamps, or professional certification programs - any setting where students produce digital work artefacts and are assessed against a defined set of learning outcomes. The contribution is not the Zigex integration specifically, but the general approach it demonstrates.

TIMEFRAME

The project runs over six months. Table 1 shows the schedule, with shaded blocks indicating active work periods for each task.

Table 1: Project Schedule

Task	**W1**	**W2**	**W3**	**W4**	**W5**	**W6**
Phase 1: Literature Review & Requirements						
Literature review & related work						
System requirements & curriculum schema						
Zigex API study & mock interface						
Phase 2: Observation Layer (SO1)						
Git hook integration & event capture						
Background file watcher						
Local event store & daily aggregation						
Phase 3: Classification & Reports (SO2)						

LLM classification pipeline						
Curriculum RAG layer						
Weekly report & gap analysis engine						
Phase 4: Testing & Write-up (SO3)						
Testing - accuracy eval.						
Supervisor review study						
Performance benchmarking						
Final report						

BUDGET ESTIMATE

Table 2 shows the estimated direct costs.

Table 2: Budget Estimate

Item	Description	Est. Cost (XAF)
Internet Connectivity	API calls, library downloads, and integration work	10,000
Printing & Binding	Final report copies for submission and oral defense	10,000
Contingency (10%)	Unforeseen costs during development or evaluation	10,000
TOTAL		**30,000**

REFERENCES

Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Unified pre-training for program understanding and generation. *Proceedings of the 2021 Conference. of the North American Chapter of the Association for Computational Linguistics*, 2655–2668. https://doi.org/10.18653/v1/2021.naacl-main.211

Coll, R. K., & Zegwaard, K. E. (2011). Pedagogy for work-integrated learning. In R. K. Coll & K. E. Zegwaard (Eds.), *International Handbook for Cooperative and Work-Integrated Education* (pp. 9–18). World Association for Cooperative Education.

Hsiao, I.-H., Sosnovsky, S., & Brusilovsky, P. (2015). Seeking their own way: A case study of personalisation in e-learning. *Computers & Education*, 83, 40–55. https://doi.org/10.1016/j.compedu.2014.12.017

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2014). The promises and perils of mining GitHub. *Proceedings of the 11th Working Conference on Mining Software Repositories*, 92–101. https://doi.org/10.1145/2597073.2597074

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Orr, K., Bennett, D., & Boud, D. (2018). Making work-integrated learning count: Student-generated criteria for assessment quality. *Higher Education Research & Development*, 37(6), 1197–1210. https://doi.org/10.1080/07294360.2018.1462305

Patrick, C.-J., Peach, D., Pocknee, C., Webb, F., Fletcher, M., & Pretto, G. (2009). *The WIL [Work Integrated Learning] report: A national scoping study*. Australian Learning and Teaching Council.

WakaTime. (2013). *WakaTime: Automatic time tracking for programs*. https://wakatime.com